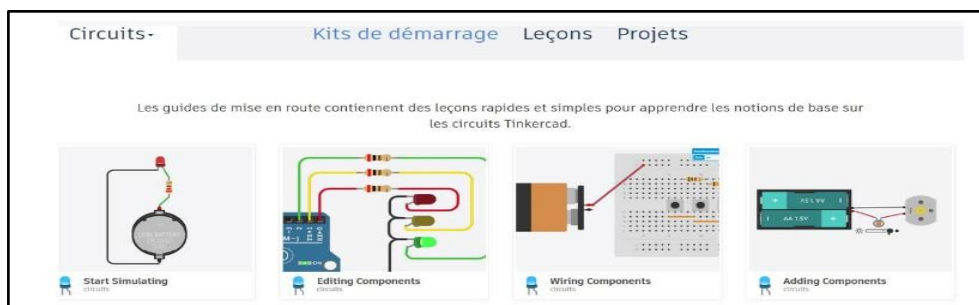


Nom Prénom :

ACTIVITE N°1

Se rendre sur <https://www.tinkercad.com> et créer un compte afin de pouvoir faire la suite des activités.

Nous allons maintenant découvrir le simulateur qui va nous accompagner sur cette partie du programme. Il y a une section «**Apprendre**» qui va nous permettre de nous familiariser avec certains composants que nous utiliserons plus tard.

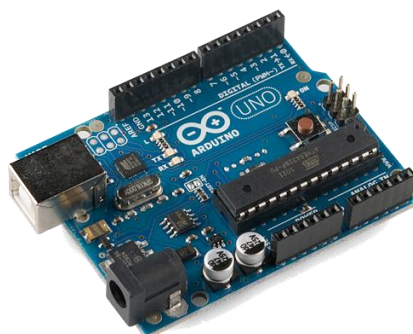


Tester et étudier ces 4 quatre tutoriels afin de bien comprendre comment fonctionne le simulateur.

I. Présentation

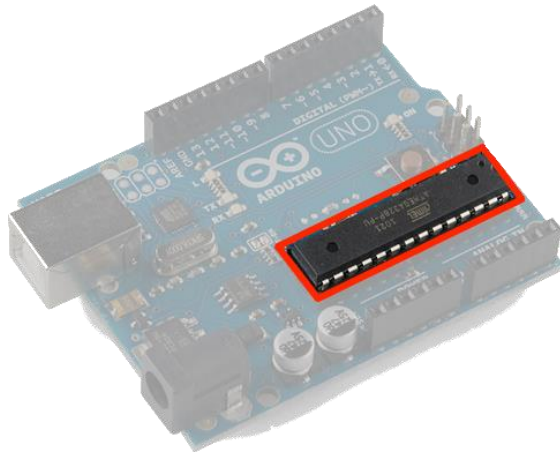
Arduino est un **circuit imprimé** en **matériel libre** sur lequel se trouve un **microcontrôleur** qui peut être programmé pour analyser et produire des **signaux électriques**.

- **Circuit imprimé** : C'est une sorte de plaque sur laquelle sont soudés plusieurs composants électroniques reliés entre eux par un circuit électrique plus ou moins compliqué. L'Arduino est donc un circuit imprimé. La photo donne une idée de la taille par rapport à la connexion USB carrée (à gauche sur la photographie, la même que sur votre imprimante par exemple).



La carte Arduino UNO

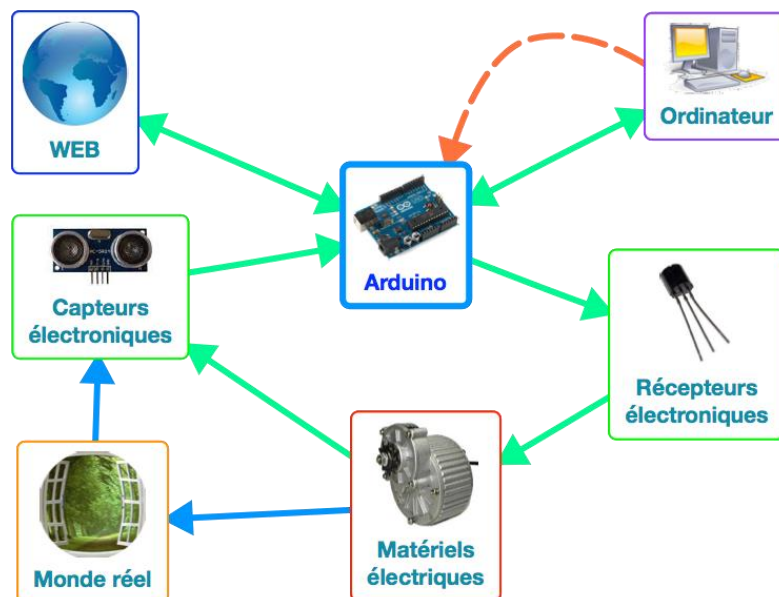
- **Microcontrôleur** : C'est le cœur de la carte Arduino. C'est une sorte d'ordinateur minuscule (mémoire morte, mémoire vive, processeur et entrées/sorties) et c'est lui que nous allons programmer. Sur la photo précédente, c'est le grand truc rectangulaire noir avec plein de pattes. Une fois lancé et alimenté en énergie, il est autonome. La force de l'Arduino est de nous proposer le microcontrôleur, les entrées/sorties, la connectique et l'alimentation sur une seule carte. La carte Arduino est construite autour d'un microcontrôleur Atmel AVR (pas toujours le même en fonction de la date de sortie de la carte) avec une capacité de mémoire de 32000 octets pour l'Arduino UNO. Soit 32 Ko, ce qui n'est vraiment pas beaucoup et qui permet pourtant de réaliser un max de projets !



Le microcontrôleur de l'Arduino

- **Signaux électriques** : L'Arduino fonctionne avec de l'électricité. Un signal électrique est un passage d'électricité dans une partie du circuit. L'Arduino est donc capable de produire ou de capter ces signaux à notre demande grâce à la programmation.

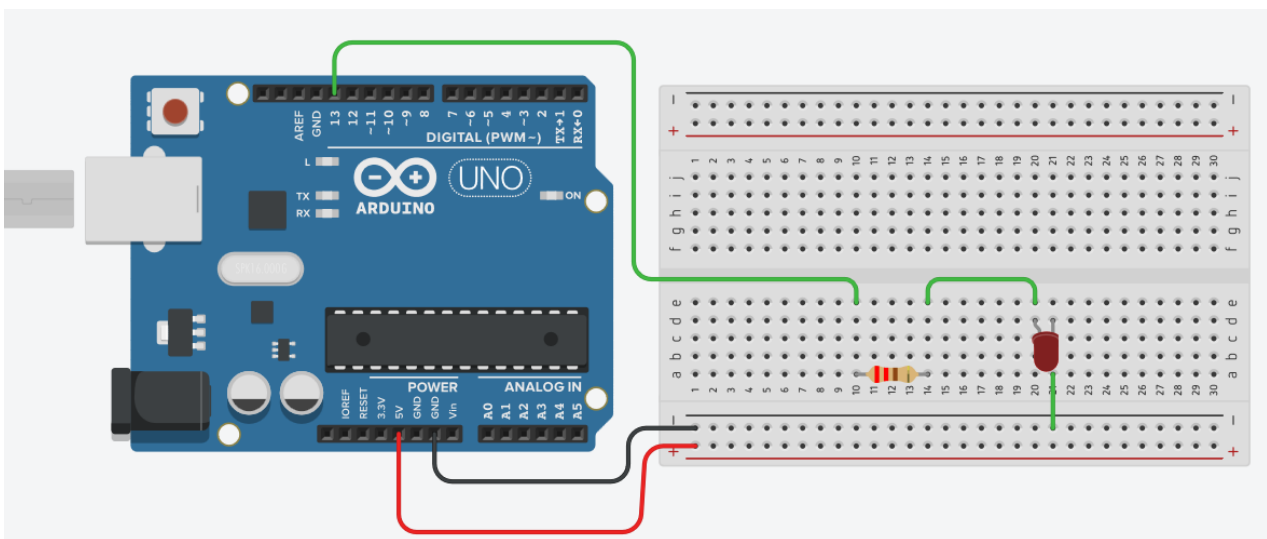
voici un schéma qui résume les principales interactions en jeu lorsque l'on programme une carte Arduino pour contrôler du matériel. Les flèches vertes indiquent la circulation des signaux électriques, la flèche orange pointillée représente l'envoi du programme vers l'Arduino et les flèches bleues les interactions avec le monde réel



II. Structure de base d'un programme Arduino

Vous allez de ce pas créer les premières lignes de votre premier programme Arduino. À partir de maintenant, il va falloir admettre que si votre programme plante, ce n'est pas la faute de l'ordinateur ou de l'Arduino, mais la vôtre. En effet, un ordinateur, même de la taille de l'Arduino, va vous obéir au doigt et à l'oeil, mais jamais ne réfléchira par lui-même. Donc si la série d'instructions que vous lui donnez l'amène à sa destruction, et bien il se détruira. Ou au mieux, il se bloquera. Nous allons donc devoir apprendre à éviter ces situations énervantes que provoquent les bugs.

Nous allons utiliser les broches (pin en anglais) 0 à 13 pour brancher les composants. En effet, on peut programmer les broches 0 à 13 pour envoyer le courant quand on le souhaite. Dans les faits, on évite d'utiliser les connexions 0 et 1, qui peuvent servir au port série (que l'on verra ultérieurement) Pour simplifier, on branchera directement le courant sur la colonne de trous où sont les composants, sans passer par les trous du bas (ou du haut). D'où le montage que l'on réalise (vous pouvez prendre n'importe quelle connexion, sauf 0 et 1) :



- La LED **ON** est verte quand l'Arduino est sous tension.
- Les LEDs **TX** et **RX** clignotent quand l'Arduino reçoit ou envoie des informations.
- La LED **L** clignote si on appuie sur le bouton reset, sinon elle ne fait rien tant qu'on ne lui a pas dit.

Dans la partie code, taper ceci:

```
void setup()
{
}

void loop()
{
}
```

Alors vous avez sûrement remarqué que les mots se colorent lorsqu'on les a saisis. C'est un petit plus très pratique qui s'appelle la **coloration syntaxique**. Elle très employée en informatique. Elle nous permet de faire la différence entre les mots clés du langage utilisé, et les autres.

Un programme informatique est une succession d'instructions que l'on donne à la machine. Ces instructions sont écrites dans un langage qui utilise des mots spécifiques. Savoir programmer, c'est déjà connaître les mots dont on a besoin.

Vous verrez bientôt dans ce chapitre l'utilité des parenthèses et des accolades. L'important pour l'instant, c'est de retenir que dans un programme écrit, toute parenthèse ou accolade ouverte DOIT être refermée. Cet oubli est souvent la cause de plantage du programme. Nous verrons plus tard qu'elles peuvent s'imbriquer les unes dans les autres. Je vous conseille donc d'ouvrir une accolade, de la refermer juste après et de revenir entre les deux pour ajouter votre code.

Qu'est-ce que la compilation ?

De le savoir ne vous changera pas la vie, mais je comprends les curieux. En fait le programme que vous écrivez n'est pas celui que recevra l'Arduino. C'est un programme adapté pour que les humains le lisent et le comprennent sans trop de difficulté. La machine, elle, ne comprend que le langage machine, une succession de 1 et de 0, qu'elle va interpréter et exécuter. La compilation est la transformation du langage de programmation utilisé par l'homme, en langage machine utilisé par la machine. C'est d'ailleurs le rôle de l'interface pour l'Arduino.

1. Allumer la LED

Les étapes du programme sont assez simples :

- a. On dit à l'Arduino que nous voulons que la connexion 13 puisse envoyer du courant (et pas en recevoir).
- b. On dit à l'Arduino d'envoyer du courant dans la connexion 13.

Exercice 1 : Tester le programme suivant :

```
void setup()
{
    pinMode(13,OUTPUT);    //Signale à l'Arduino que la connexion 13 doit
    pouvoir envoyer du courant
    digitalWrite(13,HIGH); //Demande à l'Arduino d'envoyer du courant dans la
    connexion 13
}

void loop()
{
}
}
```

2. Eteindre la LED

Exercice 2 : Modifier votre programme afin d'éteindre la LED

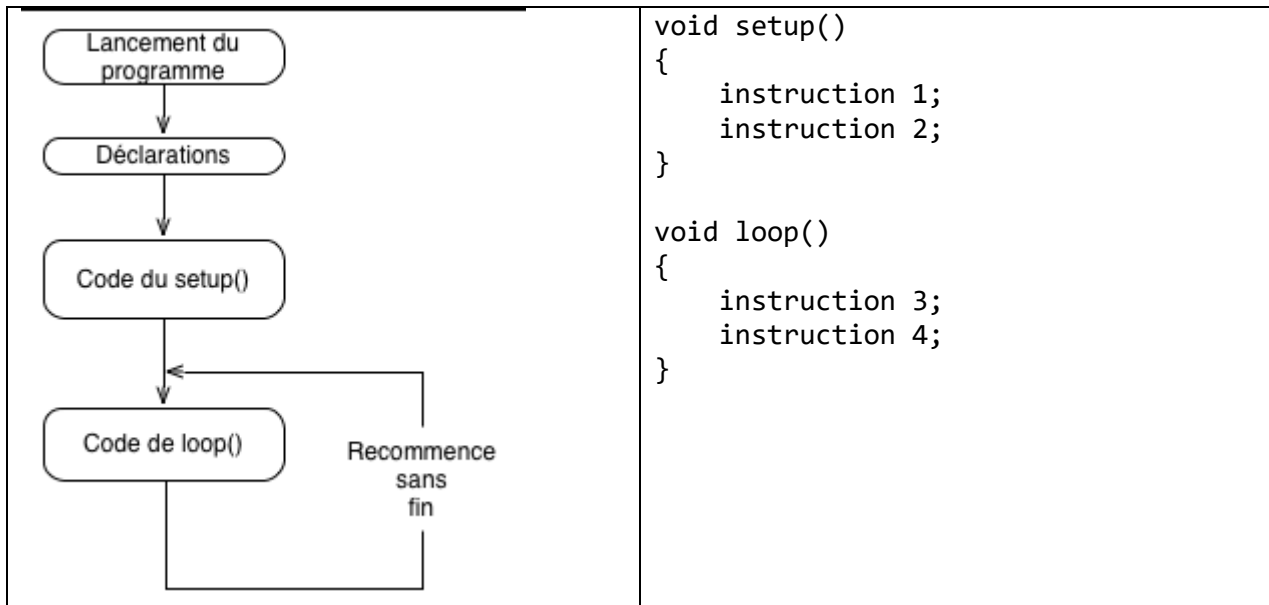
3. Faire clignoter la LED

- a. Nous indiquons à l'Arduino que la connexion 13 doit pouvoir envoyer du courant
- b. Nous indiquons d'envoyer du courant dans la connexion 13 (la diode s'allume)
- c. Nous indiquons d'arrêter d'envoyer du courant dans la connexion 13 (la diode s'éteint)
- d. Nous voulons ensuite recommencer au point numéro b. à l'infini.

Les étapes a. jusqu'à c. sont assez simples à réaliser.

Jusqu'à maintenant, nos instructions ont été mises dans les accolades de "setup". Nous dirons qu'elles sont dans le setup. Ces instructions ne sont lues et activées qu'**une seule fois**.

Les instructions mises dans les accolades de "loop" seront lues et activées **à l'infini** !



Exercice 3 : Tester le programme suivant :

```

void setup()
{
    pinMode(13,OUTPUT);
}

void loop()
{
    digitalWrite(13,HIGH);
    digitalWrite(13,LOW);
}
  
```

Exercice 4 : Quel est le problème? Que faudrait-il modifier?

4. Fréquence et période

Quand on se penche un peu sur la documentation de l'Arduino, on peut lire :

"Chaque module possède au moins un régulateur linéaire 5 V et un oscillateur à quartz 16 MHz (ou un résonateur céramique dans certains modèles)."

Ça veut dire que d'une part, la tension de sortie au niveau des connexions est de cinq volts et que notre processeur est cadencé à 16 MHz (seize méga hertz)

Le **hertz** c'est l'unité de mesure de la **fréquence** (comme le mètre est l'unité de mesure de la distance).

Et la **fréquence** c'est juste le nombre de fois par seconde qu'une action est faite. Par exemple si je tape 10 touches par seconde sur mon clavier, ma fréquence de frappe est de 10 hertz, on note 10 Hz.

Pour voir le clignotement, il faut ralentir l'Arduino.

Nous n'avons pas le choix, nous devons apprendre un nouveau mot clé : **delay()**.

delay() indique à l'Arduino d'attendre. On a juste à mettre entre parenthèses le nombre de millisecondes que l'on souhaite. Ça nécessite un léger calcul mental : 1 s = 1000 ms.

Par exemple, l'instruction suivante :

```
delay (500);
```

met le programme en pause pendant 500 ms=0,5 s donc une demi-seconde.

On utilise donc l'instruction `delay()` en écrivant la ligne de code où l'on veut que le programme fasse une pause.

Exercice 5 : Modifier le programme pour que notre LED 13 s'allume 1 seconde et s'éteigne 1 seconde.

5. Les commentaires dans le programme

Tous les programmeurs, ou codeurs, ont des conventions à respecter s'il souhaitent que leur travail soit considéré comme agréable et bien fait. Nous verrons au fil de ce cours quelques unes de ces conventions, et la première dès maintenant :

Un programme se doit d'être commenté.

Pour insérer un commentaire simple, il suffit de saisir le code `//` (double slash) et du coup, tout ce qui suivra dans la ligne sera lisible par un humain et transparent pour le programme.

Pour un bloc de commentaires (c'est-à-dire des commentaires sur plusieurs lignes à la suite) on utilise deux balises : `/*` et `*/`

Voici donc notre programme avec des commentaires :

6. En résumé

- `setup()` qui s'exécute qu'une fois ;
- `loop()` qui s'exécute à l'infini ;
- `void` (qui se met toujours devant `loop` et `setup` et dont vous comprendrez le sens plus loin dans ce cours) ;
- `pinMode()` qui permet de définir une sortie en mode envoi d'électricité ou non ;
- `digitalWrite()` qui envoie de l'électricité par une sortie ;
- `delay()` qui met le programme en pause pendant un nombre défini de millisecondes.

III. Utiliser les constantes, les variables, les conditions et le moniteur série

Nous aborderons dans ce chapitre la notion de variables et de constantes, indispensable à la programmation. Nous verrons comment les créer, les manipuler, et surtout leur intérêt dans un programme.

1. Constantes et variables

- Constantes

Vous avez remarqué que nous avons écrit le nombre 13 trois fois au total dans le programme. Ce nombre dans notre programme n'a pas de raison de changer. Il va être constant pendant toute l'exécution. Nous pourrions donc en faire une constante ! Et nous allons le faire !!

Exercice 6 : Reprenez le programme et modifiez le comme suit (je modifie les commentaires) :

```
// on définit la constante en donnant un nom et un nombre.
const int CONNEXION=13;

void setup()
{
    // on l'utilise en écrivant juste son nom
    pinMode(CONNEXION,OUTPUT);
}
void loop()
{
    digitalWrite(CONNEXION,HIGH);
    delay(1000);
    digitalWrite(CONNEXION,LOW);
    delay(1000);
}
```

La déclaration d'une constante est toujours de la forme :

```
const type NOMDELACONSTANTE=valeur;
```

- Variables

La déclaration de variable peut se faire de différentes façons. Pour des raisons de bonnes habitudes et de lisibilité, nous allons utiliser une méthode en deux temps. Je la trouve plus utile dans le cas où vous voudriez un jour programmer sur autre chose que l'Arduino et en plus elle est moins coûteuse en espace mémoire (ce qui, vous le verrez est important pour les microcontrôleurs).

- *Premier temps* : je dis à l'IDE que je réserve une place en mémoire pour une variable et je lui donne un nom (cette opération est faite une seule fois dans le programme).
- *Autres temps* : je dis à l'IDE d'affecter une valeur à la variable (cette opération peut être faite à tout moment dans le programme).

```
int nombreDeBilles;//on déclare une variable de type int nommée nombreDeBilles
nombreDeBilles=4;//on affecte le nombre 4 à la variable nombreDeBilles
nombreDeBilles=8;//on affecte le nombre 8 à la variable nombreDeBilles
```

Exercice 7 : Reprenez le programme et modifiez le comme suit. Observer le résultat

```
int tempsDePause; // déclaration de la variable tempsDePause

void setup()      // Boucle d'initialisation
{
    pinMode(13,OUTPUT); //Connexion 13 en mode envoi de courant
    tempsDePause = 1000; // initialisation à 100
}

void loop()       // Boucle infinie
{
    digitalWrite(13,HIGH); // envoi de courant dans la connexion 13
    delay(tempsDePause);   // attente d'une seconde
    digitalWrite(13,LOW);  // arrêt du courant dans la connexion 13
    delay(tempsDePause);   // attente d'une seconde
    tempsDePause = tempsDePause - 10; // Décrémentement de la variable
}
```

2. Le typage des variables et constantes

Le **typage** est obligatoire lorsqu'on définit une variable ou une constante. Dans certains langages (comme le Python par exemple), il n'est pas nécessaire car le compilateur se chargera de le faire et l'exécution du programme s'occupera des modifications éventuelles.

type	taille en byte	valeurs stockées
boolean	1	true ou false
char	1	un caractère ou un entier entre -128 et 127
unsigned char	1	un entier entre 0 et 255
byte	1	un entier entre 0 et 255
int	2	un entier entre -32768 et 32767
unsigned int	2	un entier entre 0 et 65535
float	4	un décimal, précis à 7 chiffres après la virgule

Pas d'accent, pas d'espace dans les noms de variable. Évitez en général les caractères spéciaux. Interdit aussi de commencer un nom de variable par un chiffre.

3. Portée des variables

La portée des variables (appelé aussi scope) c'est une façon commode de dire que là où on déclare une variable, c'est là où on va pouvoir l'utiliser et pas ailleurs.

Rappelez-vous que les accolades définissent des blocs de code à exécuter. Et bien si vous définissez une variable dans un bloc de code (donc entre des accolades), elle ne pourra être appelée ou modifiée QUE dans CE bloc de code.

Une fois que l'on sort des accolades du bloc, la place utilisée par la variable en mémoire est libérée. Ce qui nous laisse la possibilité de l'utiliser pour une autre variable.

4. Le moniteur série

Comme vous l'avez sûrement remarqué, l'Arduino de base n'a pas d'écran. On peut lui en ajouter un, nous aurons l'occasion de le voir plus tard, mais en attendant, on ne peut rien afficher. C'est là qu'entre en scène le **moniteur série**, qui est une fenêtre que l'on peut ouvrir en cliquant sur la loupe du logiciel Arduino. Cette fenêtre reçoit et envoie des infos sous forme de séries de bytes aux ports concernés. Donc grâce au moniteur série, l'Arduino peut (à condition d'être connecté à un PC) envoyer des infos à l'ordinateur qui va pouvoir les afficher en temps réel.

Pour ce faire on fait appel à une bibliothèque spéciale, déjà incluse dans l'IDE : la bibliothèque **Serial**.

Pour utiliser la possibilité de communication entre l'ordinateur et l'Arduino, on procède en deux étapes :

- a. On initialise la communication (ça se fait dans le `setup()`).
- b. On communique (ça se fait souvent dans la `loop()` mais possible dans le `setup()`).

Dans le programme suivant, vous allez découvrir de nouveaux mots-clés de communication comme `Serial.begin`, `Serial.println`...

Exercice 8 : Tester le programme suivant

```
void setup()
{
  Serial.begin(9600); // initialisation de la communication
  Serial.println("Communication initialisée"); // envoi d'un message
}
void loop()
{
  Serial.println("Je suis dans la boucle !"); // envoi d'un autre message
}
```

Le 9600 dans les parenthèse de l'initialisation correspond à un nombre de caractères par seconde qu'on appelle des bauds. L'Arduino peut donc envoyer un maximum de 9600 caractères par seconde à l'ordinateur dans cette configuration. Ne pas confondre avec l'unité bps, qui veut dire bits par seconde. Un caractère pour l'Arduino c'est 8 bits.

Exercice 9 : Modifier ce programme pour que le message dans la boucle ne soit envoyé que toutes les 2 secondes.

À partir de maintenant, nous appellerons **pin**, les connexions disponibles sur l'Arduino. Le pin 13 correspondra donc à la connexion 13 (ou broche 13, ou trou 13).

5. Programme « Hello Arduino World ! »

Exercice 10 : à l'ouverture du moniteur, un message qui affiche sur trois lignes "Hello Arduino World !" avec un mot par ligne (le point d'exclamation avec le dernier mot). Le message ne doit pas se répéter à l'infini.

Vous remarquerez que l'instruction `Serial.println()` affiche le texte entre guillemets et retourne à la ligne. Modifiez le programme en utilisant l'instruction `Serial.print()` donc en enlevant 'ln' à la fin. Vous verrez que les mots se collent les uns aux autres. On retient donc que :

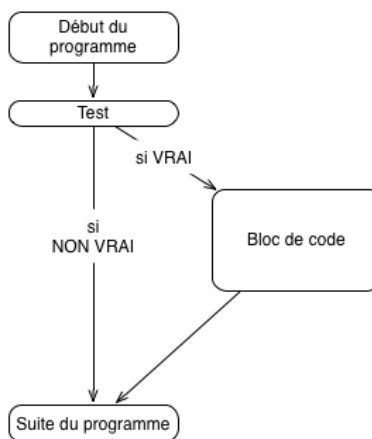
- `Serial.println()` : affiche avec retour à la ligne
- `Serial.print()` : affiche sans retour à la ligne

Exercice 11 : à l'ouverture du moniteur, un message qui affiche sur une ligne "Hello Arduino World !".
Le message ne doit pas se répéter à l'infini.

6. La condition si

La condition est un test que va réaliser le programme en se posant une question. S'il peut y répondre 'vrai' alors il exécute un bloc d'action (qui sera mis entre accolades), sinon il ne l'exécute pas.

```
debut du programme
si (test)
{
    bloc de code
}
le programme continue
```



Dans notre programme "Hello Arduino world!", nous allons ajouter un test pour vérifier si le message a bien été affiché sur la console une fois. Si oui, on ne l'affiche plus.

Exercice 12 : Tester le code suivant :

```
int affichageFait;    // déclaration de la variable

void setup()
{
    Serial.begin(9600);
    affichageFait=0;  //initialisation de la variable
}

void loop()
{
    if (affichageFait==0) //code est exécuté que si la condition est vérifiée
    {

        Serial.println("Hello");
        Serial.println("Arduino");
        Serial.println("World !");
        affichageFait=1;//on passe la variable à 1
    }
}
```

Une condition s'écrit donc avec le mot-clé **if**, puis la condition à tester est mise entre parenthèses, puis le code à exécuter entre accolades :

```
if (condition) { code à exécuter }
```

Code	Condition testée
==	égal
>=	supérieur ou égal
<=	inférieur ou égal
>	supérieur
<	inférieur
!=	différent (non égal)

IV. Les boucles et les calculs

1. Arduino compte tout seul

```
\\ Tout comme avec Python, il est possible de réaffecter les variables:  
int uneVariable;           \\ Déclaration de la variable  
uneVariable=10;            \\ Affectation de la valeur 10  
uneVariable=uneVariable+22; \\ Affectation de la valeur 10+22=32
```

Exercice 13 : Programmer l'Arduino pour qu'il compte tout seul de 1 en 1 et affiche le résultat :

Exercice 14 : Quelle est la valeur maximale atteinte ? Pourquoi ?

Exercice 15 : Modifier le programme afin que l'Arduino compte uniquement jusqu'à 20

Voici le pseudo code à utiliser:

```
déclaration de la variable compteur  
  
void setup()  
{  
    initialisation de la communication  
    initialisation de compteur à 1  
}  
  
void loop()  
{  
    si (compteur est inférieur ou égal à 20)  
    {  
        affichage de la valeur de compteur sur le moniteur  
        incrémentation de compteur  
    }  
}
```

2. La boucle FOR

La programmation nous permet de réaliser des boucles dans un programme. C'est à dire qu'on peut exécuter un bloc de code plusieurs fois avant de passer à la suite. Cette répétition ouvre un grand nombre de possibilités.

Il existe plusieurs sortes de boucles, nous les verrons progressivement. Nous allons commencer par la bouclefor. Elle permet de répéter un code un nombre de fois connu à l'avance.

Elle se présente un peu comme une condition : elle commence par un mot-clé (for), suivi de certaines choses entre parenthèses, puis des accolades qui contiennent le code à exécuter. Voici un exemple :

```
for (int t=0;t<10;t=t+1)  
{  
    code à exécuter  
}
```

Le plus difficile à comprendre se situe entre les parenthèses. Il y a trois informations importantes séparées par des points-virgules :

- a. **Initialisation** de la variable qui va servir de compteur. Dans notre exemple, on déclare et on initialise (dans le même temps) une variable t de type "int" : `int t=0` .
- b. **Condition à tester** pour continuer ou non la répétition. Ici, on exécute le code tant que t est inférieur à 10 : `t<10`.
- c. **Action sur le compteur** à chaque tour de boucle. Dans notre cas, t est augmenté de 1 à chaque passage : `t=t+1`.

Exercice 16 : Exécuter ce programme et observer le résultat

```
void setup()
{
  Serial.begin(9600);
  Serial.println("*** Debut du programme ***");
}
void loop()
{
  for (int compteur=0;compteur<=20;compteur=compteur+1)
  {
    Serial.println(compteur);
  }
}
```

Il compte bien jusqu'à 20, mais reprend ce comptage sans arrêt !

C'est normal, nous n'avons pas indiqué à Arduino qu'il ne doit le faire qu'une seule fois. Il va donc falloir créer une variable à tester pour le lui indiquer.

3. Le type boolean

On peut écrire `boolean a=1;` ou `boolean a=true;` la valeur de true est 1.

Exercice 17 : Observer le code suivant

```
boolean etat; //déclaration de la variable etat de type boolean
etat=true; //initialisation de etat à VRAI
if (etat) // test si etat est VRAI
{
  //bloc de code exécuté si etat est VRAI
}
```

Vous remarquerez que l'on n'a pas besoin d'écrire `if (etat==true)` dans la condition, il suffit d'écrire `if (etat)` qui signifie la même chose (mais les deux sont corrects).

Exercice 18 : Ecrire un programme qui permet à l'Arduino de compter jusqu'à 20 puis de s'arrêter

Exercice 19 : Ecrire un programme qui permet à l'Arduino de faire clignoter la LED un nombre n de fois.

4. TP table de multiplication

Dans ce programme, je vais vous demander d'afficher la table de multiplication par 7 des nombres de 0 à 14. Voici quelques règles à respecter :

- Le programme devra afficher la table de multiplication une seule fois, il faudra donc utiliser une condition pour vérifier si elle a été affichée.

- Pour rendre le programme facile à modifier, vous stockerez le nombre 7 dans une variable de type int. Nous apprendrons plus tard comment envoyer une information à l'Arduino depuis le moniteur, ce qui nous permettra d'afficher la table de notre choix.)
- Le moniteur devra afficher une table qui ressemble à ça :

```
*****
Table de multiplication
La table de : 7

0 x 7 = 0
1 x 7 = 7
2 x 7 = 14
3 x 7 = 21
4 x 7 = 28
5 x 7 = 35
6 x 7 = 42
7 x 7 = 49
8 x 7 = 56
9 x 7 = 63
10 x 7 = 70
11 x 7 = 77
12 x 7 = 84
13 x 7 = 91
14 x 7 = 98

*****
```

Quelques pistes utiles

- Pour afficher un saut de ligne, il suffit d'utiliser l'instruction suivante : `Serial.println()`, sans rien entre les parenthèses.
- Pour faire un calcul de multiplication dans un programme, on utilise l'étoile *. Donc "3x4" en humain s'écrit "3*4" en machine. Et la division s'écrit avec le slash / en machine.

Exercice 20 : Ecrire le programme